Pipeline de Données
Panel Administratif
2015–2025

Documentation Technique Complète

Version 2.1 — Architecture Python / Polars / MinIO

Périmètre : Indicateurs de la masse salariale des agents de l'État

Couverture temporelle : Janvier 2015 – Décembre 2025

Technologie : Python · Polars · MinIO (S3) · openpyxl

Mise à jour v2.1 : INNER JOIN F1↔F2 + filtration 124+421 dans F2 + suppression du fallback `montant_brut`

Document généré le March 30, 2026

Contents

1 Résumé exécutif **3**
- 1.1 Vue d'ensemble . . . 3
- 1.2 Livrables produits . . . 3
- 1.3 Référentiels Staging . . . 3
- 1.4 Suppressions de lignes dans le pipeline . . . 3

2 Contexte et objectifs **4**
- 2.1 Contexte . . . 4
- 2.2 Objectifs techniques . . . 4
- 2.3 Contraintes de conception . . . 4

3 Architecture technique **5**
- 3.1 Stack technologique . . . 5
- 3.2 Organisation du stockage objet . . . 5
- 3.3 Flux de données . . . 5
- 3.4 Connexion à MinIO . . . 5

4 Description des scripts **6**
- 4.1 00_upload_staging.py — Dépôt initial des sources . . . 6
  - 4.1.1 Objectif . . . 6
  - 4.1.2 Variables configurables . . . 6
  - 4.1.3 Comportement détaillé . . . 6
  - 4.1.4 Dépendances Python . . . 6
  - 4.1.5 Fichiers produits . . . 6
- 4.2 01_pre_analyse.py — Pré-analyse des colonnes . . . 6
  - 4.2.1 Objectif . . . 6
  - 4.2.2 Variables configurables . . . 6
  - 4.2.3 Logique technique . . . 7
  - 4.2.4 Analyses réalisées . . . 7
  - 4.2.5 Fichiers produits . . . 7
- 4.3 02_staging_to_bronze.py — Staging vers Bronze . . . 7
  - 4.3.1 Objectif . . . 7
  - 4.3.2 Variables configurables . . . 7
  - 4.3.3 Fonctions utilitaires . . . 7
  - 4.3.4 Logique de traitement . . . 8
  - 4.3.5 Nommage du fichier Parquet . . . 8
  - 4.3.6 Comportement en cas d'erreur . . . 8
  - 4.3.7 Fichiers produits . . . 8
- 4.4 03_bronze_to_silver.py — Bronze vers Silver . . . 8
  - 4.4.1 Objectif . . . 8
  - 4.4.2 Variables configurables . . . 8
  - 4.4.3 Fonctions utilitaires . . . 9
  - 4.4.4 Chargement des tables ANSTAT . . . 9
  - 4.4.5 Logique de traitement par période . . . 10
  - 4.4.6 Rapport de matching ANSTAT . . . 10
  - 4.4.7 Fichiers produits . . . 10
- 4.5 04_validation_silver.py — Validation qualité . . . 10
  - 4.5.1 Objectif . . . 10
  - 4.5.2 Variables configurables . . . 10
  - 4.5.3 Contrôles réalisés . . . 10
  - 4.5.4 Résumé final . . . 11
  - 4.5.5 Fichiers produits (si anomalies) . . . 11
- 4.6 05_silver_to_gold.py — Silver vers Gold . . . 11
  - 4.6.1 Objectif . . . 11
  - 4.6.2 Variables configurables . . . 11

4.6.3 Table Gold 1 : masse_salariale.parquet . . . 11
4.6.4 Table Gold 2 : effectifs.parquet . . . 11
4.6.5 Export CSV . . . 11
4.7 06_compiler_panel.py — Compilation du panel . . . 11
4.7.1 Objectif . . . 11
4.7.2 Colonnes du panel (ordre exact) . . . 11
4.7.3 Fonction construire_panel_annee(df) . . . 12
4.7.4 Fusion complète en streaming . . . 12
4.7.5 Statistiques affichées . . . 13
4.7.6 Fichiers produits . . . 13
4.8 06b_imputer_grades.py — Imputation des grades . . . 13
4.8.1 Objectif . . . 13
4.8.2 Pourquoi une étape dédiée ? . . . 13
4.8.3 Constantes . . . 13
4.8.4 Fonction extraire_grade(statut) . . . 13
4.8.5 Séquence en 6 étapes . . . 13
4.8.6 Valeurs de GRADE_SOURCE . . . 13
4.8.7 Bilan affiché . . . 13
4.9 07_calcul_indicateur.py — Calcul des indicateurs . . . 13
4.9.1 Objectif . . . 14
4.9.2 Variables configurables . . . 14
4.9.3 Composantes du salaire brut . . . 14
4.9.4 Nom du fichier de sortie (dynamique) . . . 15
4.9.5 Constantes barème . . . 15
4.9.6 Séquence complète en 12 étapes . . . 15
4.9.7 Feuilles produites . . . 16
4.9.8 Feuille MODELE_ANSTAT . . . 16
4.10 08_creation_fichier_excel_avec_sommaire.py . . . 16
4.10.1 Objectif . . . 16
4.10.2 Fichiers source et sortie . . . 17
4.10.3 Fonctions utilitaires Excel . . . 17
4.10.4 Structure du sommaire (5 sections) . . . 17
4.10.5 Feuilles de détail créées . . . 17

5 L'orchestrateur 18
5.1 Présentation . . . 18
5.2 Étapes gérées . . . 18
5.3 Arguments en ligne de commande . . . 18
5.4 Gestion des échecs . . . 18
5.5 Résumé final . . . 18

6 Guide de démarrage rapide 19
6.1 Prérequis . . . 19
6.2 Étapes d'exécution complète . . . 19
6.3 Erreurs fréquentes . . . 20
6.4 Relancer depuis un point d'échec . . . 20

7 Référence rapide 21
7.1 Séquence complète des scripts . . . 21
7.2 Colonnes du Gold (panel) . . . 21
7.3 Feuilles Excel produites par 07 . . . 22
7.4 Exemple de chargement du panel . . . 22

8 Positionnement dans le cadre GSBPM 23
8.1 Présentation du GSBPM . . . 23
8.2 Correspondance phases GSBPM ↔ scripts . . . 23
8.3 Activités transversales GSBPM . . . 24
8.3.1 Gestion des métadonnées . . . 24
8.3.2 Gestion de la qualité . . . 24

9 Glossaire 25

1. Résumé exécutif

Ce document décrit le pipeline de traitement de données qui produit les indicateurs statistiques sur les salaires des agents de la fonction publique à partir des fichiers de solde mensuels bruts (2015–2025).

1.1 Vue d'ensemble

Le pipeline transforme environ 120 fichiers Excel mensuels bruts en un panel individu × période exploitable, puis calcule les indicateurs de rémunération diffusés dans un fichier Excel structuré.

- **Staging** : dépôt initial des fichiers sources (Excel mensuels + référentiels)
- **Bronze** : données brutes converties en Parquet, tout en `String`, sans transformation
- **Silver** : données enrichies, types corrects, situations normalisées, codes ANSTAT rattachés
- **Gold** : agrégats (masse salariale, effectifs) + panel individu × période avec grades imputés

1.2 Livrables produits

1. **Panel complet** `s3://gold/panel_admin/panel_complet.parquet` et fichiers annuels `panel_YYYY.parquet` : une ligne par agent × mois, avec grade imputé, code CITP, barème salarial.
2. **Tables Gold d'agrégats** :
   - `s3://gold/panel_admin/masse_salariale.parquet`
   - `s3://gold/panel_admin/effectifs.parquet`
3. **Fichier d'indicateurs** : `indicateurs_{mode}_{periode}_{zeros}.xlsx` (multi-feuilles : grades, CITP, barèmes, multi-postes, modèle ANSTAT)
4. **Fichier avec sommaire navigable** : `indicateurs_*_SOMMAIRE.xlsx` (5 sections cliquables, feuilles de détail par grade/CITP/sexe/barème)
5. **Rapports de qualité** dans `s3://staging/panel_admin/validation/` : taux de matching ANSTAT, doublons, colonnes suspectes.

1.3 Référentiels Staging

Trois types de fichiers de référence sont déposés dans `s3://staging/panel_admin/references/` avant toute exécution du pipeline :

1. **FICHIER_ANSTAT_CODE.xlsx** — dictionnaire de codes administratifs (organisme, grade, emploi, affectation, service, fonction, poste) avec libellés normalisés pour le matching textuel.
2. **CORRESPONDANCE_GRILLE_EMPLOI.xlsx (nouveau)** — table de correspondance emploi → barème / grille salariale, utilisée pour le fuzzy join de l'étape 07.
3. Fichiers Excel mensuels déposés dans `s3://staging/panel_admin/fichiers_mensuels/`.

1.4 Suppressions de lignes dans le pipeline

Le pipeline supprime des lignes à trois étapes clés :

- **Étape 02 (Staging → Bronze)** : dans la Feuille 2, seuls les individus ayant à la fois `124_F2` ET `421_F2` non nuls sont conservés. La jointure F1↔F2 est une `INNER JOIN` — les individus absents de l'une des deux feuilles sont éliminés.
- **Étape 03 (Bronze → Silver)** : toute ligne dont `situation_normalisee` vaut `autre` est supprimée. Les situations conservées sont `en_activite`, `regul_indemnites`, `demi_solde`. Les données F2 associées à ces lignes sont également écartées.
- **Étape 07 (Calcul indicateurs)** : les lignes dont `GRADE_SOURCE` est `NF_NON_IMPUTABLE` sont exclues des calculs. Les agents de sexe inconnu sont aussi exclus.

2. Contexte et objectifs

2.1 Contexte

La production des indicateurs de la masse salariale de la fonction publique repose sur des fichiers de solde mensuels (format Excel, deux feuilles : données agents et codes de rémunération). Ces fichiers, produits depuis 2015, présentent des hétérogénéités structurelles importantes : noms de colonnes variables, lignes d'en-tête décalées, formats de date divers.

Le pipeline automatise l'intégralité de la chaîne de traitement, de l'ingestion brute jusqu'à la production des indicateurs statistiques.

2.2 Objectifs techniques

- Centraliser tous les fichiers sources sur un stockage objet partagé (MinIO) pour permettre le travail collaboratif depuis JupyterHub.
- Standardiser les données sur l'ensemble de la série temporelle malgré les hétérogénéités structurelles.
- Constituer un panel longitudinal individu $\times$ période de qualité, avec grades validés et codes CITP rattachés.
- Produire des indicateurs salariaux fiables (P25, P50, P75, min, max, moyenne) par grade, CITP, sexe, barème salarial.
- Générer un fichier Excel navigable destiné aux utilisateurs métier.

2.3 Contraintes de conception

- **Mémoire limitée** : le panel complet (120+ fichiers, plusieurs millions de lignes) est traité en streaming via `scan_parquet` / `sink_parquet` de Polars.
- **Robustesse** : chaque étape gère les erreurs fichier par fichier. Un fichier en erreur n'arrête pas le traitement des autres.
- **Traçabilité** : chaque ligne du panel conserve son `fichier_source` et son `GRADE_SOURCE` (`OBSERVE`/`IMPUTE`/`NF_NON_IMPUTABL`
- **Reproductibilité** : les paramètres de calcul (période, mode salaire, seuil de winsorisation) sont centralisés en tête de chaque script.

3. Architecture technique

3.1 Stack technologique

Composant	Rôle
Python 3.10+	Langage de traitement
Polars	Manipulation de DataFrames (plus rapide que pandas, API Lazy)
boto3	Client S3 pour MinIO
fastexcel	Lecture Excel rapide (backend Polars)
openpyxl	Écriture Excel stylisée (sortie finale)
python-dotenv	Gestion des variables d'environnement
MinIO	Stockage objet S3-compatible (auto-hébergé)
JupyterHub	Environnement d'exécution partagé

3.2 Organisation du stockage objet

Structure des buckets MinIO

```
staging/ panel_admin/ fichiers_mensuels/ ← Fichiers Excel sources (MMYYYY.xlsx) references/ FICHIER_ANSTAT_CODE.xlsx
CORRESPONDANCE_GRILLE_EMPLOI.xlsx ← NOUVEAU pre_analyse/ ← Rapports 01 (CSV) validation/ ← Rapports 04 + 03 (CSV qualité)
exports_gold/ ← Fichiers Excel finaux + CSV exports
bronze/ panel_admin/ MMYYYY.parquet ← Un fichier par mois (ex:  012015.parquet)
silver/ panel_admin/ YYYY.parquet ← Un fichier par année (ex:  2020.parquet)
gold/ panel_admin/ masse_salariale.parquet ← Agrégats masse salariale effectifs.parquet ← Agrégats effectifs
panel_YYYY.parquet ← Panel annuel (ex:  panel_2020.parquet) panel_complet.parquet ← Panel toutes années fusionné
```

3.3 Flux de données



3.4 Connexion à MinIO

Chaque script supporte deux modes de connexion :

Configuration locale vs JupyterHub

```
Depuis la machine locale MINIO_ENDPOINT = "http://192.168.1.230:30137" MINIO_ACCESS_KEY = "datalab-team" MINIO_SECRET_KEY
= "minio-datalabteam123"
Depuis JupyterHub (décommenter)  MINIO_ENDPOINT = "http://minio.mon-namespace.svc.cluster.local:80"
```

Les scripts des étapes 03 à 07 utilisent des paramètres de retry renforcés : 5 tentatives maximum, mode adaptatif, `connect_timeout=30s`, `read_timeout=120s`.

4. Description des scripts

4.1 00_upload_staging.py — Dépôt initial des sources

Ce script doit être lancé **une seule fois** avant toute exécution du pipeline. Il transfère les fichiers sources depuis le poste local vers MinIO staging, et crée les quatre buckets s'ils n'existent pas.

4.1.1 Objectif

Déposer les fichiers sources sur MinIO afin que toute l'équipe puisse travailler depuis staging, sans dépendance aux chemins locaux.

4.1.2 Variables configurables

Variable	Description
`DOSSIER_EXCEL`	Chemin local vers le dossier contenant les fichiers Excel mensuels (parcours récursif `**/*.xlsx`). Défaut : `consolidation_solde_2015_2025/01_data_sources/fichiers_solde_mensuels`
`FICHIER_ANSTAT`	Chemin local vers le fichier ANSTAT_CODE. Défaut : `.../FICHIER_ANSTAT_CODE_2025.xlsx`
`FICHIER_CORRESPONDANCE`	**(Nouveau)** Chemin local vers la table de correspondance emploi → barème. Défaut : `.../CORRESPONDANCE_GRILLE_EMPLOI.xlsx`
`MINIO_ENDPOINT`	URL du serveur MinIO
`MINIO_ACCESS_KEY`	Clé d'accès MinIO
`MINIO_SECRET_KEY`	Clé secrète MinIO

4.1.3 Comportement détaillé

Création automatique des buckets. Le script vérifie l'existence des quatre buckets (`staging`, `bronze`, `silver`, `gold`) via `head_bucket` et les crée si nécessaire.

Upload des fichiers mensuels. Tous les fichiers `.xlsx` trouvés récursivement dans `DOSSIER_EXCEL` sont uploadés vers `s3://staging/panel_admin/fichiers_mensuels/{nom_fichier}`. Un compteur OK/erreurs est affiché à la fin.

Upload du fichier ANSTAT. Le fichier `FICHIER_ANSTAT` est uploadé vers `s3://staging/panel_admin/references/{nom}`.

Upload de la table de correspondance. (Nouveau) `CORRESPONDANCE_GRILLE_EMPLOI.xlsx` est uploadé vers `s3://staging/panel_admin/references/CORRESPONDANCE_GRILLE_EMPLOI.xlsx`. Si le fichier est absent localement, un message indique le chemin attendu sans arrêter le script.

4.1.4 Dépendances Python

```
pip install boto3 python-dotenv
```

Note : Polars n'est pas requis pour cette étape.

4.1.5 Fichiers produits

- `s3://staging/panel_admin/fichiers_mensuels/*.xlsx`
- `s3://staging/panel_admin/references/FICHIER_ANSTAT_CODE.xlsx`
- `s3://staging/panel_admin/references/CORRESPONDANCE_GRILLE_EMPLOI.xlsx`

4.2 01_pre_analyse.py — Pré-analyse des colonnes

4.2.1 Objectif

Analyse la structure des fichiers Excel en staging et produit des rapports CSV sur les variations de colonnes d'une période à l'autre. Étape exploratoire, sans modification des données.

4.2.2 Variables configurables

Variable	Description
`BUCKET`	Bucket source. Défaut : `staging`
`PREFIX_MENSUELS`	Préfixe des fichiers mensuels. Défaut : `panel_admin/fichiers_mensuels`
`PREFIX_SORTIE`	Préfixe de sortie des rapports. Défaut : `panel_admin/pre_analyse`

4.2.3 Logique technique

Listage et tri. Les fichiers `.xlsx` sont listés via paginateur S3 et triés par nom (ordre alphabétique = ordre chronologique, car les noms sont au format `MMYYYY`).

Lecture des en-têtes. Chaque fichier est téléchargé dans un fichier temporaire (`download_file` puis `NamedTemporaryFile`), puis lu avec :

```
f1 = pl.read_excel(tmp_path, sheet_id=1, read_options="n_rows": 0) f2 = pl.read_excel(tmp_path, sheet_id=2,
read_options="n_rows": 0)
```

Le paramètre `read_options={"n_rows": 0}` (et non `engine_options`) charge uniquement les en-têtes sans les données, pour des raisons de performance.

Extraction des codes numériques F2. Les colonnes dont le nom est purement numérique sont identifiées par :

```
codes_num_f2 = [c for c in f2.columns if str(c).strip().isdigit()]
```

Gestion des erreurs. Un fichier en erreur est ajouté à la liste `erreurs` et n'arrête pas l'analyse des fichiers suivants.

4.2.4 Analyses réalisées

- Colonnes apparues / disparues entre deux périodes consécutives (Feuille 1)
- Codes numériques apparus / disparus entre deux périodes consécutives (Feuille 2)
- Statistiques globales (min/max/moyenne du nombre de colonnes par fichier, codes F2 min/max)

4.2.5 Fichiers produits

Fichier	Contenu
`colonnes_f1_uniques.csv`	Liste de toutes les colonnes F1 distinctes sur l'ensemble des périodes
`codes_f2_uniques.csv`	Liste de tous les codes numériques F2 distincts
`stats_par_periode.csv`	Nombre de colonnes F1/F2/codes par période
`changements_f1.csv`	Colonnes apparues / disparues F1 (si changements détectés)
`changements_codes_f2.csv`	Codes numériques apparus / disparus F2 (si changements détectés)

Tous ces fichiers sont déposés dans `s3://staging/panel_admin/pre_analyse/`.

4.3 02_staging_to_bronze.py — Staging vers Bronze

4.3.1 Objectif

Convertit chaque fichier Excel mensuel en un fichier Parquet Bronze. Bronze = données brutes, tout en `String` (`Utf8`), sans transformation métier. Un fichier Parquet par période est produit.

4.3.2 Variables configurables

Variable	Description
`BUCKET_STAGING`	Bucket source. Défaut : `staging`
`BUCKET_BRONZE`	Bucket cible. Défaut : `bronze`
`PREFIX_MENSUELS`	Défaut : `panel_admin/fichiers_mensuels`
`PREFIX_BRONZE`	Défaut : `panel_admin`

4.3.3 Fonctions utilitaires

normaliser_nom_colonne(nom)

Normalise un nom de colonne brut en identifiant standardisé :

1. Supprime les préfixes du type `SHEET1.` via regex `^[A-Z_]+[0-9]+\.`
2. Passe en majuscules
3. Remplace les caractères `-`, `/`, `_`, espace par `_`
4. Remplace les points par `_`
5. Normalisation Unicode NFKD + suppression des combinants (suppression des accents)
6. Supprime tous les caractères non alphanumériques sauf `_`
7. Déduplique les `_` consécutifs
8. Supprime les `_` en début/fin

deduplication_noms(noms)

Rend une liste de noms de colonnes unique en suffixant les doublons avec `_2`, `_3`, etc. Exemple : `[NOM, NOM, NOM]` → `[NOM, NOM_2, NOM_3]`.

detecter_entete(fichier, sheet_id)
Détecte automatiquement la ligne d'en-tête contenant "matricule" :

1. Lit les 20 premières lignes sans en-tête (`header_row=None, n_rows=20`)
2. Parcourt chaque ligne à la recherche du mot "matricule" (insensible à la casse)
3. Renvoie le numéro de ligne (0-indexé) si trouvé
4. Défaut : ligne 1 pour Feuille 1, ligne 0 pour Feuille 2

4.3.4 Logique de traitement

Pour chaque fichier Excel mensuel :

1. Téléchargement dans un fichier temporaire (`download_file`)
2. Détection de la ligne d'en-tête F1 et F2 via `detecter_entete()`
3. Lecture Feuille 1 avec `header_row=skip_f1`
4. Normalisation et déduplication des noms de colonnes F1
5. Lecture Feuille 2 avec `header_row=skip_f2`
6. Jointure F2 sur la clé **matricule‖code_organisme** :
   - La colonne clé F2 est détectée par regex `MATRICULE.*\|\|`
   - Renommée en `CLE_UNIQUE_F2`
   - Toutes les autres colonnes F2 reçoivent le suffixe `_F2` (ex: `124_F2`, `228_F2`)
   - **Filtrage F2** : avant la jointure, on ne conserve dans F2 que les individus ayant à la fois `124_F2` ET `421_F2` non nuls. Ces deux codes sont indispensables au calcul du salaire brut.
   - **INNER JOIN** sur `CLE_UNIQUE` ou `MATRICULE` selon disponibilité — seuls les individus présents dans les deux feuilles sont conservés.
7. Ajout de deux colonnes de métadonnées :
   - `PERIODE` : nom du fichier sans extension (format `MMYYYY`)
   - `FICHIER_SOURCE` : nom du fichier avec extension
8. Cast de toutes les colonnes en `Utf8` (String)
9. Écriture Parquet avec compression Snappy : `s3://bronze/panel_admin/{periode}.parquet`
10. Suppression du fichier temporaire

4.3.5 Nommage du fichier Parquet

Le nom du fichier Parquet Bronze est le nom du fichier Excel source sans l'extension, soit le format `MMYYYY` (ex: `012015.parquet`).

4.3.6 Comportement en cas d'erreur

Un fichier en erreur incrémente le compteur `nb_err` et n'arrête pas le traitement des autres fichiers. Le fichier temporaire est supprimé même en cas d'erreur (bloc `finally`).

4.3.7 Fichiers produits

`s3://bronze/panel_admin/MMYYYY.parquet` — un fichier par mois.

4.4 03_bronze_to_silver.py — Bronze vers Silver

4.4.1 Objectif

Applique la logique métier sur les données brutes Bronze : mapping des colonnes vers des noms standardisés, normalisation des situations administratives, enrichissement avec les codes ANSTAT, cast en types numériques, suivi du taux de matching par période.

4.4.2 Variables configurables

Variable	Description
`BUCKET_STAGING`	`staging`
`BUCKET_BRONZE`	`bronze`
`BUCKET_SILVER`	`silver`
`PREFIX_BRONZE`	`panel_admin`
`PREFIX_SILVER`	`panel_admin`
`PREFIX_REFS`	`panel_admin/references`
`PREFIX_VALID`	`panel_admin/validation`
`CODES_A_SURVEILLER`	`["CODE_ORGANISME", "CODE_GRADE", "CODE_EMPLOI"]` — codes dont le taux de matching est loggué en temps réel

4.4.3 Fonctions utilitaires

normaliser_pour_matching(texte)

Normalisation textuelle utilisée pour le fuzzy join sur les libellés ANSTAT :

1. Remplace les tirets par des espaces
2. Remplace les caractères `.,;:()\[]/` par des espaces
3. Remplace les apostrophes (droite, courbe, unicode U+2019) par des espaces
4. Passage en majuscules
5. Normalisation Unicode NFKD + suppression des combinants
6. Normalisation des espaces multiples

mapper_colonnes(noms_colonnes)

Mappe chaque colonne brute vers un nom standardisé via 26 patrons regex. Le tableau suivant liste le mapping complet :

Nom standardisé	Patterns regex
cle_unique	MATRICULE.*CODE.*ORGANISME, MATRICULE.*||.*CODE
matricule	^MATRICULE$, ^MATRIC$
nom	^NOM$, ^NOM_PRENOM$, ^NOM_ET_PRENOM$
date_naissance	DATE.*NAISSANCE, DATE_NAISS, NAISSANCE
sexe	^SEXE$
situation_matrimoniale	SITUATION.*MATRIMONIALE, STATUT.*MATRIMONIAL
nombre_enfant	NOMBRE.*ENFANT, NBR.*ENFANT
situation	^SITUATION$, SITUATION_ADMINISTRATIVE
date_debut_situation	DATE.*DEBUT.*SITUATION, DATE_SITUATION
montant_brut	MONTANT_BRUT, SALAIRE_BRUT, REMUNERATION_BRUTE, ^BRUT$
montant_net	MONTANT_NET, SALAIRE_NET, REMUNERATION_NETTE, ^NET$
retenue_pension	RETENUE.*PENSION, ^PENSION$, COTISATION.*PENSION
impot	^IMPOT$, ^IGR$, IMPOT.*REVENU
charge_patronale	CHARGE.*PATRONALE, CHARGES.*PATRONALES
organisme	^ORGANISME$, MINISTERE, DIRECTION
lieu_affectation	LIEU.*AFFECTATION, ^AFFECTATION$
service	^SERVICE$
emploi	^EMPLOI$, ^CORPS$, CORPS.*EMPLOI
fonction	^FONCTION$
grade	CLASSE.*ECHELON, CLASSE_ECHELON
statut_fonctionnaire	^GRADE$
poste	^POSTE$, LIBELLE.*POSTE
prise_service	PRISE.*SERVICE, DATE.*PRISE.*SERVICE
date_retraite	DATE.*RETRAITE
age_retraite	AGE.*RETRAITE
mois_annee	MOIS.*ANNEE, ^PERIODE$

Si aucun patron ne correspond, le nom normalisé (minuscules) est conservé.

normaliser_situation(situation)

Mappe la valeur brute de situation vers une catégorie normalisée :

- `en_activite` : EN ACTIVITE, ACTIVITE
- `regul_indemnites` : REGUL. INDEMNITES, REGUL INDEMNITES, REGULARISATION INDEMNITES
- `demi_solde` : DEMI-SOLDE, DEMI SOLDE, DEMISOLDE, 1/2 SOLDE
- `autre` : toute valeur non reconnue (y compris null, vide)

charger_table_codes(...)

Charge un onglet du fichier ANSTAT, identifie la colonne libellé par pattern, la normalise pour le matching, et renvoie un dictionnaire avec la table, le nom de la colonne code, et la colonne de jointure source.

4.4.4 Chargement des tables ANSTAT

Huit tables de référence sont chargées depuis le fichier ANSTAT :

Code	Onglet	Colonnes extra
CODE_AFFECTATION	*lieu affectation*	CODE_RÉGION, CODE_SITE, CODE_DRB, CODE_DÉPARTEMENT, MONTANT TRANSPORT
CODE_ORGANISME	*ORGANISME_OK*	CODE_ÉTABLISSEMENT, CODE_ACCT
CODE_POSITION_SOLDE	*CODE_SITUATION_SOLDE*	—
CODE_EMPLOI	*HISTORIQUE_ECHELLES_CORPS*	GRADE_ADMINISTRATIF_ASSOCIE, Code_CITP, Metier_CITP
CODE_SERVICE	*SERVICE*	—
CODE_FONCTION	*FONCTION*	—
CODE_GRADE	*GRADE*	—
CODE_POSTE	*LIBELLE POSTE*	—

4.4.5 Logique de traitement par période

Pour chaque fichier Bronze mensuel :

1. Lecture du Parquet depuis MinIO Bronze
2. Mapping des colonnes via `mapper_colonnes()`, avec déduplication des noms standards si collision
3. **Forçage de mois_annee** depuis le nom du fichier source (plus fiable que la valeur interne — certains fichiers Bronze contiennent un `mois_annee` erroné car il s'agit de doublons)
4. Normalisation des situations et filtrage : les lignes `autre` sont **supprimées**
5. Enrichissement ANSTAT : pour chaque code, jointure gauche sur libellé normalisé ; calcul du taux de matching
6. Cast numérique des montants et des codes numériques (colonnes commençant par un chiffre)
7. Accumulation dans `frames_par_annee`

4.4.6 Rapport de matching ANSTAT

Un rapport `taux_matching_anstat_par_periode.csv` est sauvegardé dans `s3://staging/panel_admin/validation/`. Il contient, pour chaque période, le taux de matching (en %) pour chacun des 8 codes ANSTAT.

Les codes `CODES_A_SURVEILLER` (CODE_ORGANISME, CODE_GRADE, CODE_EMPLOI) voient leur taux loggué en temps réel pendant l'exécution.

4.4.7 Fichiers produits

`s3://silver/panel_admin/YYYY.parquet` — un fichier par année.

4.5 04_validation_silver.py — Validation qualité

4.5.1 Objectif

Contrôles qualité sur les tables Silver. Étape **optionnelle** dans l'orchestrateur : un échec n'arrête pas le pipeline.

4.5.2 Variables configurables

Variable	Description
COLONNES_CLES	Colonnes dont la complétude est contrôlée toutes années confondues : `matricule`, `nom`, `montant_brut`, `montant_net`, `CODE_ORGANISME`, `CODE_GRADE`, `CODE_EMPLOI`, `situation_normalisee`, `mois_annee`
COLONNES_NA_SURVEY	Colonnes dont le taux de NA est suivi par année : `matricule`, `nom`, `montant_brut`, `montant_net`, `organisme`, `grade`, `emploi`, `lieu_affectation`, `service`

4.5.3 Contrôles réalisés

1. **Complétude des colonnes clés** (toutes années confondues). Alerte si taux de complétion $< 95\%$.
2. **Colonnes suspectes** (NA variable selon l'année). Seuil : taux NA $> 95\%$ au max ET $< 5\%$ au min (variation > 90 points entre années). Si des colonnes suspectes sont détectées, le rapport `na_par_annee_colonnes_surveillees.csv` est sauvegardé.
3. **Doublons matricule $\times$ mois_annee**. Le rapport `doublons_matricule_periode.csv` est sauvegardé si des doublons existent.
4. **Cohérence des montants** (net $>$ brut) : comptage des lignes incohérentes.
5. **Distribution des situations administratives** : répartition `en_activite` / `regul_indemnites` / `demi_solde`.

4.5.4 Résumé final

À l'issue de la validation, le script affiche le nombre d'agents uniques (matricules distincts), la plage temporelle, et le nombre de périodes distinctes présentes dans le Silver.

4.5.5 Fichiers produits (si anomalies)

- `s3://staging/panel_admin/validation/na_par_annee_colonnes_surveillees.csv`
- `s3://staging/panel_admin/validation/doublons_matricule_periode.csv`

4.6 05_silver_to_gold.py — Silver vers Gold

4.6.1 Objectif

Produit deux tables Gold d'agrégats à partir des tables Silver annuelles.

4.6.2 Variables configurables

Variable	Description
`BUCKET_SILVER`	`silver`
`BUCKET_GOLD`	`gold`
`BUCKET_STAGING`	`staging`
`KEY_GOLD_MS`	`panel_admin/masse_salariale.parquet`
`KEY_GOLD_EFF`	`panel_admin/effectifs.parquet`
`PREFIX_EXPORTS`	`panel_admin/exports_gold`

4.6.3 Table Gold 1 : masse_salariale.parquet

Agrégation par `mois_annee` × `annee` × `mois` × `code_organisme` × `organisme` × `situation_normalisee`.

Colonnes produites :

Colonne	Définition
`nb_lignes`	Nombre de lignes dans le groupe
`nb_agents_uniques`	Nombre de matricules distincts
`masse_brute`	Somme des montants bruts
`masse_nette`	Somme des montants nets
`salaire_brut_moyen`	Moyenne du montant brut
`salaire_net_moyen`	Moyenne du montant net
`salaire_brut_mediane`	Médiane du montant brut (P50)
`salaire_net_mediane`	Médiane du montant net (P50)

Note : seules les lignes où `montant_brut` ET `montant_net` sont non nuls participent à cette agrégation. `code_organisme` vaut `NON_CODE` si null, `organisme` vaut `NON_IDENTIFIE` si null.

4.6.4 Table Gold 2 : effectifs.parquet

Agrégation par `mois_annee` × `annee` × `mois` × `situation_normalisee` × `code_organisme`.

Colonnes produites :

Colonne	Définition
`nb_lignes`	Nombre de lignes dans le groupe
`nb_agents_uniques`	Nombre de matricules distincts
`nb_hommes`	Nombre de lignes où `sexe == "1"`
`nb_femmes`	Nombre de lignes où `sexe == "2"`

4.6.5 Export CSV

Deux exports CSV sont déposés dans `s3://staging/panel_admin/exports_gold/` :

- `masse_salariale_par_periode_organisme.csv`
- `effectifs_par_periode.csv`

4.7 06_compiler_panel.py — Compilation du panel

4.7.1 Objectif

Construit le panel individu × période à partir des tables Silver annuelles. Produit des fichiers annuels et un fichier complet en streaming.

4.7.2 Colonnes du panel (ordre exact)

Groupe	Colonne	Type
Identité	matricule	Utf8
	nom	Utf8
	sexe	Utf8
	date_naissance	Utf8
	situation_matrimoniale	Utf8
	nombre_enfant	Utf8
Position	situation_normalisee	Utf8
	CODE_ORGANISME	Utf8
	organisme	Utf8
	CODE_EMPLOI	Utf8
	emploi	Utf8
	Code_CITP	Utf8
	Metier_CITP	Utf8
	CODE_GRADE	Utf8
	grade	Utf8
	statut_fonctionnaire	Utf8
	CODE_FONCTION	Utf8
	fonction	Utf8
	CODE_SERVICE	Utf8
	service	Utf8
	CODE_AFFECTATION	Utf8
	lieu_affectation	Utf8
	CODE_POSTE	Utf8
	prise_service	Utf8
	date_retraite	Utf8
	age_retraite	Utf8
Temporel	mois_annee	String (brut Silver)
	annee	Int32
	mois	Int32
Salaires	montant_brut	Float64
	montant_net	Float64
	retenue_pension	Float64
	impot	Float64
	charge_patronale	Float64
Traçabilité	fichier_source	Utf8

Ces deux colonnes sont ajoutées par l'étape **06b** (`06b_imputer_grades.py`), pas par 06. Elles sont placées juste après `statut_fonctionnaire` dans l'ordre final.

4.7.3 Fonction construire_panel_annee(df)

Pour chaque table Silver annuelle :

1. Extraction de `annee` et `mois` depuis `mois_annee`. Deux formats supportés :
   - Format MMYYYY (6 caractères) : `annee = slice(2,4)`, `mois = slice(0,2)`
   - Format YYYY-MM : `annee = slice(0,4)`, `mois = slice(5,2)`
2. Sélection des colonnes présentes dans `COLONNES_PANEL` (ordre préservé, colonnes absentes ignorées)
3. Déduplication sur `matricule` × `mois_annee` (garde la **première** occurrence)
4. Tri par `matricule` → `annee` → `mois`

4.7.4 Fusion complète en streaming

Le fichier `panel_complet.parquet` est produit par :

1. Téléchargement de tous les fichiers `panel_YYYY.parquet` dans un répertoire temporaire (`TemporaryDirectory`)
2. `pl.scan_parquet(chemins_locaux)` : lecture en mode lazy, sans charger les données en mémoire
3. `.sort([...]).sink_parquet(chemin, compression="snappy")` : écriture en streaming, jamais tout en RAM

4. Upload du fichier local vers MinIO Gold

Cette approche permet de traiter le panel complet (millions de lignes) sans surcharger la mémoire.

4.7.5 Statistiques affichées

Par année : `nb_obs`, `nb_individus`, `brut_moyen`, `brut_mediane`.

4.7.6 Fichiers produits

- `s3://gold/panel_admin/panel_YYYY.parquet` (un par année)
- `s3://gold/panel_admin/panel_complet.parquet`

4.8 06b_imputer_grades.py — Imputation des grades

4.8.1 Objectif

Calcule les grades manquants ou invalides par imputation statistique en cascade sur la colonne `emploi`, puis met à jour tous les fichiers Gold (panel annuels + panel complet).

4.8.2 Pourquoi une étape dédiée ?

L'imputation de niveau P3 (global, toutes années) est d'autant plus fiable qu'elle s'appuie sur l'intégralité de l'historique. Travailler année par année donnerait des lookups instables pour les emplois rares.

4.8.3 Constantes

```
GRADES_VALIDES = A1..A7, B1..B6, C1..C5, D1..D3  21 grades GRADES_INVALIDES = ["B7", "C7", "D7", "A", "B", "C", "D"]
```

4.8.4 Fonction extraire_grade(statut)

Extrait le grade depuis `statut_fonctionnaire` via l'expression régulière `Cat[eé]gorie\s+([ABCD][0-9]*)` (insensible à la casse). Exemples : "Catégorie A3" → `A3`, "Catégorie b2" → `B2`.

Si aucun grade n'est trouvé, retourne `"NF"`.

Note : les grades NF (non-fonctionnaires ou valeur non reconnue) restent `NF` et ne sont pas imputés.

4.8.5 Séquence en 6 étapes

Étape 1. Chargement du `panel_complet.parquet`.

Étape 2. Extraction du grade brut via `extraire_grade()`. Bilan : grades valides / invalides / NF affiché.

Étape 3. Imputation en cascade (si `n_invalides > 0`) :

- Extraction de `_ANNEE` (slice 2:4 de mois_annee) et `_MOIS_NUM` (slice 0:2) pour les lookups
- Construction de la base des grades valides uniquement
- **Lookup P1** : `GRADE.mode().first()` groupé par `emploi` × `_ANNEE` × `_MOIS_NUM`
- **Lookup P2** : `GRADE.mode().first()` groupé par `emploi` × `_ANNEE`
- **Lookup P3** : `GRADE.mode().first()` groupé par `emploi` (global, toutes années)
- Application de la cascade : si grade invalide, tenter P1 → P2 → P3, conserver l'invalide si aucun lookup n'aboutit

Étape 4. Réordonnancement : `GRADE` et `GRADE_SOURCE` placés juste après `statut_fonctionnaire`.

Colonnes temporaires supprimées : `_ANNEE`, `_MOIS_NUM`, `GRADE_P1`, `GRADE_P2`, `GRADE_P3`, `GRADE_SOURCE_TMP`.

Étape 5. Mise à jour des fichiers Gold annuels : jointure sur `(matricule, mois_annee)` pour injecter `GRADE` et `GRADE_SOURCE`.

Étape 6. Mise à jour du `panel_complet.parquet`.

4.8.6 Valeurs de GRADE_SOURCE

Valeur	Signification
`OBSERVE`	Grade extrait directement de `statut_fonctionnaire`, valide
`IMPUTE`	Grade remplacé par le mode statistique de l'emploi (P1/P2/P3)
`NF_NON_IMPUTABLE`	Aucun grade valide trouvé ni imputé (non-fonctionnaire ou emploi sans historique de grade valide)

4.8.7 Bilan affiché

Pourcentage OBSERVE / IMPUTE / NF_NON_IMPUTABLE + nombre de grades encore invalides après imputation.

4.9 07_calcul_indicateur.py — Calcul des indicateurs

4.9.1 Objectif

Produit les indicateurs statistiques complets sur les rémunérations à partir des fichiers Gold panel. Requiert que `06b_imputer_grades.py` ait été exécuté au préalable (colonnes GRADE et GRADE_SOURCE obligatoires).

4.9.2 Variables configurables

Variable	Défaut	Description
`MODE_SALAIRE_BRUT`	`True`	`True` = agrégation par `cle_unique` (salaire brut) ; `False` = agrégation par `matricule` (revenu salarial)
`PERIODE_RECENTE`	`True`	`True` = années 2024–2025 ; `False` = 2015 → dernière année
`INCLURE_ZEROS`	`True`	`True` = inclure salaires = 0 ou NA
`SEUIL_WINS`	`0.01`	Seuil de winsorisation P1/P99. Mettre `None` pour désactiver
`MIN_OBS_WINS`	`100`	Effectif minimum par cellule mois × grade × CITP pour appliquer la winsorisation CITP
`KEY_CORRESPONDANCE`	`panel_admin/references/CORRESPONDANCE_GRILLE_EMPLOI.xlsx`	Clé S3 de la table barème

4.9.3 Composantes du salaire brut

```
COMPOSANTES_SALAIRE = ["124_f2", "228_f2", "271_f2", "421_f2", "425_f2"]
```

Ces colonnes correspondent aux codes de rémunération de la Feuille 2, avec le suffixe `_F2` ajouté lors de l'étape 02. Le salaire brut est calculé par somme horizontale de ces composantes.

En raison du filtrage F2 (124 et 421 obligatoires) et de l'INNER JOIN appliqués à l'étape 02, chaque individu du Gold est garanti avoir les composantes F2. Si aucune composante n'est trouvée, le script lève une erreur explicite plutôt qu'un fallback silencieux.

4.9.4 Nom du fichier de sortie (dynamique)

```
indicateurs_mode_label_periode_label_zeros_label.xlsx  Exemples :  indicateurs_salaire_brut_2024_2025_avec_zeros.xlsx
indicateurs_revenu_salarial_2015_recent_sans_zeros.xlsx
```

4.9.5 Constantes barème

```
GRADES_PRIMAIRE = {"B3", "C3", "D1", "D2"}
ORDRE_MODELE_ANSTAT = [ ("TOUS SECTEURS CONFONDUS", "_GLOBAL_"), ("SECTEUR EDUCATION", None), ("Primaire", "ENS SEC &
PRIM_PRIM"), ("Secondaire", "ENS SEC & PRIM_SEC"), ("Supérieur", "ENS SUP"), ("SECTEUR SANTE", None), ("Cadres supérieurs",
"Cadre-Sup-Santé Generaliste & Specialiste"), ("Personnels Techniques", "Tech-Santé"), ("AUTRES SECTEURS", None),
("Para-militaire / Douanes / Eaux & Forêts", "police"), ("Barème Général", "Barême Général"), ("PERSONNELS SOUS STATUT
PARTICULIER", None), ("Magistrat", "MAGISTRAT "), ("Corps préfectoral", "CORPS PREF"), ("Corps diplomatique", "CORPS
DIPL"), ("Greffier", "GREFFE"), ]
```

4.9.6 Séquence complète en 12 étapes

Étape 1 — Chargement

Chargement des fichiers `panel_YYYY.parquet` (pas le `panel_complet`, pour éviter de le charger deux fois). Concaténation diagonale en mémoire.

Étape 2 — Vérification GRADE / GRADE_SOURCE

Si l'une des deux colonnes est absente, le script lève une `RuntimeError` et demande d'exécuter `06b` d'abord.

Étape 3 — Extraction ANNEE / MOIS

Depuis `mois_annee` en format MMYYYY : `ANNEE = slice(2,4)`, `MOIS_NUM = slice(0,2)`. Filtre sur `ANNEES_ANALYSE`.

Étape 4 — Calcul SALAIRE_BRUT

Somme horizontale des composantes F2 présentes (`124_f2`, `228_f2`, `271_f2`, `421_f2`, `425_f2`). Si aucune composante n'est trouvée (situation anormale après INNER JOIN), le script lève une `RuntimeError` explicite.

Étape 5 — Standardisation SEXE_STD

SEXE_STD	Valeurs reconnues
`Homme`	MASCULIN, M, H, HOMME, MALE, 1
`Femme`	FEMININ, F, FEMME, FEMALE, 2, FÉMININ, FEMININE
`Inconnu`	Tout autre → **exclu des calculs**

Étape 6 — Détection multi-postes

Comptage du nombre de lignes par `matricule` × `mois_annee` → colonne `NB_POSTES`.

Étape 7 — Filtrage grades valides

Filtre sur `GRADE.is_in(GRADES_VALIDES)`. Si `INCLURE_ZEROS=False`, filtre additionnel `SALAIRE_BRUT > 0`. Les `NF_NON_IMPUTABLE` sont exclus.

Étape 7b — Fuzzy join emploi → barème

(Nouveau) Trois fonctions :

- `charger_table_bareme()` : télécharge `CORRESPONDANCE_GRILLE_EMPLOI.xlsx` depuis staging, normalise `emploi_ref` pour le matching, retourne une table `(emploi_ref_norm, bareme, grade_ref)`. Si le fichier est absent : retourne `None`, `BAREME_DISPONIBLE=False`, feuilles barème ignorées.
- `fuzzy_join_bareme(df, table_join, col_emploi)` : ajoute les colonnes `bareme` et `grade_ref` via jointure gauche sur libellé normalisé. Avertissement si taux $< 20\%$.
- `ajouter_bareme_detail(df, col_grade)` : scinde `ENS SEC & PRIM` en `ENS SEC & PRIM_PRIM` (grades B/C/D dans `GRADES_PRIMAIRE`) et `ENS SEC & PRIM_SEC` (grades A).

Étape 8 — Agrégation REVENU_TOTAL

Agrégation par `(ANNEE, MOIS_NUM, MOIS, cle_unique|matricule)` :

- `REVENU_TOTAL` = somme de `SALAIRE_BRUT`
- `GRADE_PRINCIPAL` : grade du poste le plus élevé (valeur ordinale : A=80+n, B=70+n, C=60+n, D=50+n)
- `GRADE_SOURCE_PRINCIPAL` : source du grade principal
- `SEXE_STD`, `Code_CITP`, `Metier_CITP`, `bareme`, `bareme_detail`

Étape 8b — Winsorisation REVENU_TOTAL

(Nouveau) Appliquée en mémoire uniquement (le Gold n'est pas modifié) :

- Si n_obs dans le groupe `mois` × `grade` × `CITP` $\geq$ `MIN_OBS_WINS` : clip à P1/P99 du groupe CITP
- Sinon : clip à P1/P99 du groupe `mois` × `grade` (repli)
- Crée la colonne `REVENU_TOTAL_W` utilisée pour les indicateurs

Étape 9 — Indicateurs REVENU TOTAL

Fonction `compute_indicators()` : effectif, moyenne, médiane, min, max, P25, P75 par groupe. Groupements produits :

- Par grade
- Par grade × sexe
- Par CITP × métier
- Par barème (si `BAREME_DISPONIBLE`)
- Par barème × grade (si `BAREME_DISPONIBLE`)
- Par barème × sexe (si `BAREME_DISPONIBLE`)

Étape 10 — Indicateurs SALAIRE BRUT (pivot grade)

(Nouveau) Calcul sur les lignes individuelles (non agrégées par agent) :

- Winsorisation identique à 8b mais sur `SALAIRE_BRUT`
- Moyenne par `(ANNEE, MOIS_NUM, MOIS, GRADE)` → pivot grade → `brut_moyen_wide`
- Médiane par `(ANNEE, MOIS_NUM, MOIS, GRADE)` → pivot grade → `brut_median_wide`

Étape 11 — Multi-postes

Effectifs par `GRADE_PRINCIPAL` × `NB_POSTES` (pivot sur `1/2/3/4/5+`). Distribution globale mono/multi-postes avec revenu moyen.

Étape 12 — Export Excel

Toutes les tables sont écrites dans un fichier Excel multi-feuilles avec style d'en-tête bleu et gel de la première ligne.

4.9.7 Feuilles produites

Feuille	Contenu
`REVENU_Grade_Detail`	Effectif, moy, méd, min, max, P25, P75 par `ANNEE` × `MOIS` × `GRADE_PRINCIPAL`
`REVENU_Grade_Sexe`	Idem par `GRADE` × `SEXE_STD`
`REVENU_CITP_Detail`	Idem par `Code_CITP` × `Metier_CITP` (si CITP disponible)
`SALAIRE_BRUT_Moyen`	Salaire brut moyen par `ANNEE` × `MOIS` (pivot grade)
`SALAIRE_BRUT_Median`	Salaire brut médian par `ANNEE` × `MOIS` (pivot grade)
`MULTI_Par_Grade_NbPostes`	Effectifs par `GRADE` × `NB_POSTES` (pivot 1/2/3/4/5+)
`MULTI_Distribution`	Distribution globale mono/multi-postes avec revenu moyen
`REVENU_Bareme_Detail`	Indicateurs par barème / grille salariale (si `BAREME_DISPONIBLE`)
`REVENU_Bareme_Grade`	Indicateurs par barème × grade (si `BAREME_DISPONIBLE`)
`REVENU_Bareme_Sexe`	Indicateurs par barème × sexe (si `BAREME_DISPONIBLE`)
`MODELE_ANSTAT`	Format hiérarchique min/moyen/max par secteur (si barème disponible)

4.9.8 Feuille MODELE_ANSTAT

Format calqué sur le modèle statistique de l'ANSTAT :

- **Ligne 1** : titre fusionné A1:D1, fond bleu foncé (`1F4E79`), "MODÈLE DE DONNÉES DE SALAIRE DES FONCTIONNAIRES POUR {annee}"
- **Ligne 2** : en-têtes (fond bleu `2E75B6`) : salaire brut minimum / moyen / maximum
- **TOUS SECTEURS CONFONDUS** : fond jaune `FFF2CC`, valeurs globales
- **Titres secteur** (SECTEUR EDUCATION, SECTEUR SANTE, etc.) : fond bleu clair `D6E4F0`, cellule fusionnée A:D
- **Sous-groupes** : alternance blanc/bleu très clair `FFFFFF`/`EBF3FB`, indentation, format `#,##0`
- Largeurs : colonne A = 55, colonnes B-D = 22

Basé sur `SALAIRE_BRUT` ligne par ligne (pas sur le `REVENU_TOTAL` agrégé par agent).

4.10 08_creation_fichier_excel_avec_sommaire.py

4.10.1 Objectif

Lit le fichier d'indicateurs produit par 07 et génère un fichier Excel navigable avec un sommaire à 5 sections et des feuilles de détail.

Les trois paramètres `MODE_SALAIRE_BRUT`, `PERIODE_RECENTE` et `INCLURE_ZEROS` doivent être **identiques** à ceux utilisés lors de l'exécution de `07_calcul_indicateur.py`, afin que le script retrouve le bon fichier source.

4.10.2 Fichiers source et sortie

- **Source** : `s3://staging/panel_admin/exports_gold/indicateurs_{mode}_{periode}_{zeros}.xlsx`
- **Sortie** : `s3://staging/panel_admin/exports_gold/indicateurs_{mode}_{periode}_{zeros}_SOMMAIRE.xlsx`

4.10.3 Fonctions utilitaires Excel

Fonction	Rôle
`style_cell(cell, fill, font, align)`	Applique fill, font et alignment en une seule ligne
`merge_titre(ws, texte, row, n_cols, fill, font)`	Crée un titre fusionné stylisé sur `n_cols` colonnes
`lien_interne(ws, row, col, texte, feuille_cible)`	Crée un hyperlien Excel interne : `=HYPERLINK("#FEUILLE!A1","texte")`
`ecrire_entetes(ws, entetes, row)`	Ligne d'en-tête stylisée (fond bleu, police blanche)
`ecrire_df_dans_feuille(ws, df, start_row=4)`	Écrit un DataFrame à partir de la ligne 4 (lignes 1-3 réservées : titre, lien retour, vide)
`ajouter_lien_retour(ws, row=2)`	Lien `<- Retour au sommaire` en ligne 2
`ajuster_largeurs(ws, {col_index: width})`	Définit les largeurs de colonnes par index
`extract_grand_groupe(code_citp)`	Extrait le premier chiffre d'un code CITP (grand groupe 1-9)

4.10.4 Structure du sommaire (5 sections)

Section 1 — CITP. En-têtes : Code, Grand Groupe CITP, Effectif total, Nb métiers. 9 grands groupes (1 = Directeurs...9 = Professions élémentaires). Lien vers feuille `CITP_GG{n}`.

Section 2 — Grade. En-têtes : Grade, Description (Catégorie A/B/C/D), Effectif total, Revenu moyen (pondéré). Lien vers feuille `GRADE_{grade}`.

Section 3 — Sexe. En-têtes : Sexe, Description, Effectif total, Revenu moyen (pondéré). Lien vers feuille `SEXE_{Homme|Femme}`.

Section 4 — Multi-postes. En-têtes : Grade, Description, Mono-postes (NB_POSTES=1), Multi-postes (NB_POSTES$\geq$2). Lien vers feuille `MULTI_{grade}`.

Section 5 — Barème (nouveau, si `BAREME_DISPONIBLE`). En-têtes : Barème / Grille salariale, Effectif total, Revenu moyen, Détail grade (lien), Détail sexe (lien). En bas de section : lien vers `MODELE_ANSTAT`.

4.10.5 Feuilles de détail créées

Feuilles	Contenu
`CITP_GG1` ... `CITP_GG9`	Données CITP filtrées par grand groupe
`GRADE_A1` ... `GRADE_D3`	Indicateurs filtrés par grade
`SEXE_Homme`, `SEXE_Femme`	Indicateurs filtrés par sexe
`MULTI_A1` ... `MULTI_D3`	Multi-postes filtrés par grade
`BAREME_{nom}`	Indicateurs par barème (1 feuille par barème)
`BAREME_G_{nom}`	Indicateurs barème $\times$ grade
`BAREME_S_{nom}`	Indicateurs barème $\times$ sexe
`MODELE_ANSTAT`	Copie directe depuis le fichier 07

Chaque feuille de détail comporte en ligne 1 un titre fusionné, en ligne 2 un lien "$\leftarrow$ Retour au sommaire", et les données à partir de la ligne 4.

5. L'orchestrateur

5.1 Présentation

`orchestrateur.py` lance l'ensemble du pipeline ou une sélection d'étapes. Il gère l'ordre d'exécution, la durée, et l'arrêt en cas d'échec.

L'étape `00_upload_staging.py` est intentionnellement exclue de l'orchestrateur car elle ne doit être exécutée qu'une seule fois (dépôt initial). Lancez-la manuellement avant d'appeler l'orchestrateur.

5.2 Étapes gérées

ID	Script	Optionnelle ?
01	`01_pre_analyse.py`	Non
02	`02_staging_to_bronze.py`	Non
03	`03_bronze_to_silver.py`	Non
04	`04_validation_silver.py`	**Oui**
05	`05_silver_to_gold.py`	Non
06	`06_compiler_panel.py`	Non
06b	`06b_imputer_grades.py`	Non
07	`07_calcul_indicateur.py`	Non
08	`08_creation_fichier_excel_avec_sommaire.py`	Non

5.3 Arguments en ligne de commande

```
python orchestrateur.py  pipeline complet python orchestrateur.py -depuis 03  reprend à partir de 03 python
orchestrateur.py -etapes 06b 07 08  étapes spécifiques python orchestrateur.py -dry-run  affiche le plan sans exécuter
python orchestrateur.py -depuis 07 -dry-run  plan partiel python orchestrateur.py -repertoire /mon/chemin  scripts dans un
autre dossier
```

Argument	Description
`-depuis ID`	Reprend à partir de l'étape ID (incluse). IDs valides : 01, 02, 03, 04, 05, 06, 06b, 07, 08
`-etapes ID...`	Lance uniquement les étapes listées (ordre du pipeline préservé)
`-dry-run`	Affiche le plan d'exécution sans lancer aucun script
`-repertoire DIR`	Répertoire contenant les scripts. Défaut : dossier de l'orchestrateur (`__file__`)

5.4 Gestion des échecs

Pour chaque étape, deux types d'issues distingués :

- **ÉCHEC** : le script s'est terminé avec un code retour non nul (erreur Python non interceptée, sys.exit(1), etc.)
- **EXCEPTION** : le sous-processus n'a pas pu être lancé (fichier script introuvable, permission refusée, etc.)

Comportement selon le type d'étape :

Type	Issue	Comportement
Obligatoire	ÉCHEC	Arrêt du pipeline
Obligatoire	EXCEPTION	Arrêt du pipeline
Optionnelle (04)	ÉCHEC	Pipeline continue
Optionnelle (04)	EXCEPTION	Pipeline continue

5.5 Résumé final

À la fin de l'exécution, l'orchestrateur affiche :

- Le statut (succès / échec) de chaque étape avec sa durée
- Les étapes non lancées (suite à un arrêt anticipé)
- La durée totale du pipeline
- La commande pour reprendre depuis le premier échec : `python orchestrateur.py -depuis {id_premier_echec}`

6. Guide de démarrage rapide

6.1 Prérequis

- Python 3.10+ avec les dépendances installées : `pip install polars fastexcel boto3 python-dotenv openpyxl`
- Serveur MinIO accessible (local ou JupyterHub)
- Fichiers sources disponibles localement :
  - Dossier des fichiers Excel mensuels
  - `FICHIER_ANSTAT_CODE.xlsx`
  - `CORRESPONDANCE_GRILLE_EMPLOI.xlsx`

6.2 Étapes d'exécution complète

Étape 1 — Configurer les chemins locaux dans 00.

Ouvrir `00_upload_staging.py` et modifier les trois variables :

```
DOSSIER_EXCEL = "chemin/vers/fichiers_excel/" FICHIER_ANSTAT = "chemin/vers/FICHIER_ANSTAT_CODE.xlsx"
FICHIER_CORRESPONDANCE = "chemin/vers/CORRESPONDANCE_GRILLE_EMPLOI.xlsx"
```

Étape 2 — Configurer la connexion MinIO.

Dans chaque script, vérifier les valeurs de `MINIO_ENDPOINT`, `MINIO_ACCESS_KEY`, `MINIO_SECRET_KEY`. Décommenter le bloc JupyterHub si nécessaire.

Étape 3 — Déposer les fichiers sources (une seule fois).

```
python 00_upload_staging.py
```

Vérifier que les compteurs `OK / erreurs` sont cohérents.

Étape 4 — (Facultatif) Pré-analyser la structure.

```
python 01_pre_analyse.py
```

Consulter les rapports dans `s3://staging/panel_admin/pre_analyse/` pour identifier les variations de colonnes entre périodes.

Étape 5 — Lancer le pipeline complet.

```
python orchestrateur.py
```

Ou, si l'étape 01 a déjà été exécutée manuellement :

```
python orchestrateur.py -depuis 02
```

Étape 6 — Vérifier la qualité (Silver).

Consulter le rapport de matching dans `s3://staging/panel_admin/validation/taux_matching_anstat_par_periode.csv`. Un taux de matching faible sur `CODE_ORGANISME` indique des libellés non reconnus dans le fichier ANSTAT.

Étape 7 — Vérifier le panel Gold.

Consulter les fichiers dans `s3://gold/panel_admin/` et les statistiques affichées par `06_compiler_panel.py`.

Étape 8 — Configurer les paramètres d'indicateurs.

Ouvrir `07_calcul_indicateur.py` et ajuster :

```
MODE_SALAIRE_BRUT = True  ou False pour revenu salarial PERIODE_RECENTE = True  ou False pour la série complète
INCLURE_ZEROS = True  ou False SEUIL_WINS = 0.01  ou None pour désactiver
```

Étape 9 — Relancer les étapes 07 et 08 seulement.

```
python orchestrateur.py -etapes 07 08
```

Étape 10 — Récupérer le fichier Excel final.

Télécharger depuis `s3://staging/panel_admin/exports_gold/indicateurs_*_SOMMAIRE.xlsx`.

6.3 Erreurs fréquentes

Erreur	Cause et solution
`Fichier ANSTAT introuvable dans s3://staging/...`	Le fichier ANSTAT n'a pas été uploadé. Vérifier le chemin dans `FICHIER_ANSTAT` et relancer `00_upload_staging.py`.
`Colonnes GRADE et/ou GRADE_SOURCE absentes du Gold`	L'étape 06b n'a pas encore été exécutée. Lancer `python orchestrateur.py -depuis 06b`.
`Fichier introuvable : s3://staging/.../indicateurs_*.xlsx`	Les paramètres `MODE_SALAIRE_BRUT` / `PERIODE_RECENTE` / `INCLURE_ZEROS` dans 08 diffèrent de ceux utilisés dans 07. Les aligner.
`Aucune donnée pour les années [2024, 2025]`	Les données ne couvrent pas les années sélectionnées. Passer `PERIODE_RECENTE = False`.
`RuntimeError : Aucune composante de salaire`	Les colonnes F2 (124_f2, 228_f2, etc.) sont absentes du Gold. Vérifier que l'étape 02 a bien été exécutée après la modification (INNER JOIN + filtration 124+421). Relancer le pipeline depuis 02.
`Taux matching barème < 20%`	Peu d'emplois reconnus dans `CORRESPONDANCE_GRILLE_EMPLOI.xlsx`. Vérifier les libellés d'emploi dans ce fichier.
`ConnectionRefusedError / EndpointResolutionError`	MinIO inaccessible. Vérifier `MINIO_ENDPOINT` (local vs JupyterHub).
`NoSuchBucket`	Buckets non créés. Relancer `00_upload_staging.py` qui les crée automatiquement.

6.4 Relancer depuis un point d'échec

Si l'orchestrateur s'arrête sur l'étape 03 :

```
Corriger le problème, puis : python orchestrateur.py -depuis 03
```

Si seuls les indicateurs doivent être recalculés (paramètres modifiés) :

```
python orchestrateur.py -etapes 07 08
```

7. Référence rapide

7.1 Séquence complète des scripts

Script	Buckets	Résultat
00_upload_staging.py	local → staging	Fichiers Excel + référentiels dans staging
01_pre_analyse.py	staging → staging	Rapports CSV de structure des colonnes
02_staging_to_bronze.py	staging → bronze	MMYYYY.parquet (tout String)
03_bronze_to_silver.py	bronze + staging → silver	YYYY.parquet (enrichi, typé)
04_validation_silver.py	silver → staging	Rapports qualité CSV (optionnel)
05_silver_to_gold.py	silver → gold + staging	masse_salariale.parquet, effectifs.parquet, CSV exports
06_compiler_panel.py	silver → gold	panel_YYYY.parquet, panel_complet.parquet
06b_imputer_grades.py	gold → gold	Mise à jour GRADE + GRADE_SOURCE dans tous les fichiers Gold
07_calcul_indicateur.py	gold + staging → staging	indicateurs_*.xlsx
08_creation_fichier_excel_avec_sommaire.py	staging → staging	indicateurs_*_SOMMAIRE.xlsx

7.2 Colonnes du Gold (panel)

Le tableau suivant liste toutes les colonnes du panel Gold après exécution complète du pipeline (étapes 06 + 06b) :

Colonne	Type	Origine
matricule	Utf8	F1
nom	Utf8	F1
sexe	Utf8	F1
date_naissance	Utf8	F1
situation_matrimoniale	Utf8	F1
nombre_enfant	Utf8	F1
situation_normalisee	Utf8	03 (calcul)
CODE_ORGANISME	Utf8	ANSTAT (03)
organisme	Utf8	F1
CODE_EMPLOI	Utf8	ANSTAT (03)
emploi	Utf8	F1
Code_CITP	Utf8	ANSTAT (03)
Metier_CITP	Utf8	ANSTAT (03)
CODE_GRADE	Utf8	ANSTAT (03)
grade	Utf8	F1
statut_fonctionnaire	Utf8	F1
GRADE	Utf8	**06b** (OBSERVE/IMPUTE/NF)
GRADE_SOURCE	Utf8	**06b**
CODE_FONCTION	Utf8	ANSTAT (03)
fonction	Utf8	F1
CODE_SERVICE	Utf8	ANSTAT (03)
service	Utf8	F1
CODE_AFFECTATION	Utf8	ANSTAT (03)
lieu_affectation	Utf8	F1
CODE_POSTE	Utf8	ANSTAT (03)
prise_service	Utf8	F1

Colonne	Type	Origine
date_retraite	Utf8	F1
age_retraite	Utf8	F1
mois_annee	Utf8	Nom de fichier (03)
annee	Int32	06 (calcul)
mois	Int32	06 (calcul)
montant_brut	Float64	F1
montant_net	Float64	F1
retenue_pension	Float64	F1
impot	Float64	F1
charge_patronale	Float64	F1
fichier_source	Utf8	02 (métadonnée)
124_F2...N_F2	Float64	F2 (via 02)
CODE_RÉGION	Utf8	ANSTAT (03)
CODE_SITE	Utf8	ANSTAT (03)
CODE_DRB	Utf8	ANSTAT (03)
CODE_DÉPARTEMENT	Utf8	ANSTAT (03)
MONTANT TRANSPORT	Utf8	ANSTAT (03)
CODE_ÉTABLISSEMENT	Utf8	ANSTAT (03)
CODE_ACCT	Utf8	ANSTAT (03)
GRADE_ADMINISTRATIF_ASSOCIE	Utf8	ANSTAT (03)

7.3 Feuilles Excel produites par 07

Voir section 4.9 (tableau des feuilles produites).

7.4 Exemple de chargement du panel

Charger le panel en Python

```
import polars as pl, io, boto3 from botocore.client import Config
s3 = boto3.client("s3", endpoint_url="http://192.168.1.230:30137", aws_access_key_id="datalab-team",
aws_secret_access_key="minio-datalabteam123", config=Config(signature_version="s3v4"), region_name="us-east-1",
verify=False)
Charger une seule année buf = io.BytesIO() s3.download_fileobj("gold", "panel_admin/panel_2023.parquet", buf) buf.seek(0)
panel = pl.read_parquet(buf)
Charger toutes les années frames = [] for annee in range(2015, 2026):  buf = io.BytesIO() s3.download_fileobj("gold",
f"panel_admin/panel_annee.parquet", buf) buf.seek(0) frames.append(pl.read_parquet(buf)) panel = pl.concat(frames,
how="diagonal")
Exemple :  brut moyen par organisme et mois panel.group_by(["annee", "mois", "CODE_ORGANISME"]).agg(
pl.col("montant_brut").mean().alias("brut_moyen"), pl.col("matricule").n_unique().alias("nb_agents"), ).sort(["annee",
"mois"])
```

8. Positionnement dans le cadre GSBPM

8.1 Présentation du GSBPM

Le **Generic Statistical Business Process Model** (GSBPM, version 5.2) est le cadre de référence international pour la production de statistiques officielles, développé par l'UNECE et adopté par l'ensemble des instituts nationaux de statistique (INSEE, ONS UK, US Census Bureau, Eurostat, etc.). Il décompose toute chaîne de production statistique en huit phases séquentielles, augmentées d'activités transversales (gestion des métadonnées, gestion de la qualité).

Ces deux cadres ne sont **pas des alternatives** — ils opèrent à des niveaux différents et se complètent :

- **GSBPM** : cadre de *processus métier* — décrit *quoi faire* (collecter, intégrer, éditer, imputer, agréger, diffuser). Agnostique techniquement.
- **Architecture médaillon** : patron d'*implémentation* — décrit *comment organiser le stockage* en couches de maturité progressive (Staging → Bronze → Silver → Gold).

Le pipeline Panel Admin met en œuvre les deux simultanément : la médaillon comme organisation du lac de données MinIO, et une logique GSBPM implicite dans l'enchaînement des scripts.

8.2 Correspondance phases GSBPM ↔ scripts

Le tableau ci-dessous détaille la correspondance entre chaque phase du GSBPM et les scripts du pipeline, ainsi que l'état des données produit à chaque étape.

Phase GSBPM	Description	Script(s)	État
Phases 1–3 *Spécification* *Conception* *Construction*	Hors pipeline — conception des fichiers de solde, nomenclature ANSTAT, grilles salariales, schéma MinIO.	*(hors pipeline)*	—
Phase 4 *Collecte*	Transfert des fichiers Excel mensuels et des référentiels depuis le poste local vers MinIO staging. Création automatique des quatre buckets si absents.	`00_upload_staging.py` `01_pre_analyse.py`	Staging
Phase 5.1 *Intégration*	Conversion Excel → Parquet (tout en `String`). Jointure F1↔F2 sur `CLE_UNIQUE` (INNER JOIN). Filtrage des individus sans codes `124_F2`/`421_F2`. Normalisation des noms de colonnes.	`02_staging_to_bronze.py`	Bronze
Phase 5.2 *Classification*	Fuzzy join ANSTAT pour rattacher les codes : `CODE_ORGANISME`, `CODE_EMPLOI`, `Code_CITP`, `CODE_GRADE`, `CODE_AFFECTATION`, `CODE_REGION`, etc. Normalisation Unicode pour absorber les variations orthographiques.	`03_bronze_to_silver.py`	Silver
Phase 5.3 *Contrôle & validation*	Taux de matching ANSTAT, détection des doublons, colonnes suspectes. Rapports déposés dans `staging/.../validation/`. Suppression des lignes `situation_normalisee = autre`.	`03_bronze_to_silver.py` `04_validation_silver.py`	Silver
Phase 5.4 *Édition & imputation*	Imputation du grade en cascade : P1 = mode emploi×mois, P2 = emploi×année, P3 = emploi série entière. Traçabilité via `GRADE_SOURCE` (`OBSERVE` / `IMPUTE` / `NF_NON_IMPUTABLE`).	`06b_imputer_grades.py`	Gold
Phase 5.5 *Dérivation*	Construction du panel individu × période. Calcul de `annee`, `mois`, `REVENU_TOTAL` = `montant_brut` + composantes F2 (`124_F2`, `421_F2`, etc.).	`05_silver_to_gold.py` `06_compiler_panel.py`	Silver Gold
Phase 5.6 *Agrégation*	Tables d'agrégats par grade, CITP, sexe et mois : `masse_salariale.parquet` et `effectifs.parquet`. Couche Gold analytique, distincte du panel individuel.	`05_silver_to_gold.py`	Gold
Phase 5.7 *Finalisation*	Consolidation du panel complet (`panel_complet.parquet`) et des fichiers annuels (`panel_YYYY.parquet`). Le Gold n'est jamais modifié en aval (la winsorisation est appliquée en mémoire uniquement).	`06_compiler_panel.py`	Gold

(suite page suivante)

Phase GSBPM	Description	Script(s)	État
Phase 6 *Analyse*	Indicateurs statistiques (P25, médiane, P75, min, max, moyenne) par grade × CITP × sexe × barème. Winsorisation P1/P99 en mémoire. Exclusion des `NF_NON_IMPUTABLE` et des agents de sexe inconnu.	`07_calcul_indicateur.py`	Gold (lecture seule)
Phase 7 *Diffusion*	Fichiers Excel multi-feuilles (`indicateurs_*.xlsx`) et fichier avec sommaire navigable à 5 sections (`indicateurs_*_SOMMAIRE.xlsx`). Dépôt dans `staging/.../exports_gold/`.	`07_calcul_indicateur.py` `08_creation_fichier_excel_avec_sommaire.py`	Staging (exports)
Phase 8 *Évaluation*	**Non formalisé.** À mettre en place : comparaison édition N vs $N-1$, revue des taux de matching ANSTAT, revue des grades imputés, bilan des fichiers en erreur (étape 02).	*(à créer)*	—

8.3 Activités transversales GSBPM

Le GSBPM définit deux activités qui s'appliquent à **toutes les phases** simultanément.

8.3.1 Gestion des métadonnées

Le pipeline assure une traçabilité partielle :

- `fichier_source` : nom du fichier Excel d'origine, propagé jusqu'au Gold (phase 5.1).
- `GRADE_SOURCE` : origine du grade (`OBSERVE` / `IMPUTE` / `NF_NON_IMPUTABLE`), ajouté en phase 5.4.
- `PERIODE` : mois et année de la donnée (format `MMYYYY`).

Aucun des objets MinIO ne porte de métadonnées de production au sens GSBPM : horodatage d'upload, identifiant de run, résultat du contrôle qualité. Recommandation : lors de chaque `put_object`, ajouter les métadonnées S3 suivantes via le paramètre `Metadata` de boto3 :

- `x-run-timestamp` : date et heure d'exécution
- `x-script-name` : nom du script ayant produit l'objet
- `x-qa-status` : résultat du dernier contrôle qualité (`OK` / `WARNING` / `FAILED`)

8.3.2 Gestion de la qualité

Les mécanismes de qualité actuels :

- Taux de matching ANSTAT par période (étape 04).
- Compteur d'erreurs fichier par fichier à l'étape 02 — un fichier en erreur n'arrête pas le traitement.
- Rapports CSV de validation dans `s3://staging/panel_admin/validation/`.
- Exclusion explicite des agents `NF_NON_IMPUTABLE` et de sexe inconnu avant calcul des indicateurs.

Ce dispositif couvre les phases 5.3 et 5.4 du GSBPM. La phase 8 (évaluation inter-éditions) reste à formaliser.

9. Glossaire

ANSTAT Système national de codification administrative. Le fichier `FICHIER_ANSTAT_CODE.xlsx` contient les dictionnaires de codes pour les organismes, grades, emplois, affectations, services, fonctions et postes.

Barème salarial Grille de rémunération associée à un corps d'emploi (ex: Barème Général, ENS SUP, MAGISTRAT). La table `CORRESPONDANCE_GRILLE_EMPLOI.xlsx` fait correspondre chaque libellé d'emploi à son barème.

Bronze Couche de données brutes : données issues des fichiers Excel, converties en Parquet sans aucune transformation métier, tout en `String`. Philosophie : "Bronze = fidèle à la source".

CITP Classification Internationale Type des Professions. Le code CITP est rattaché via la table ANSTAT (`CODE_EMPLOI`). Le premier chiffre identifie le grand groupe (1 = cadres, 2 = professions intellectuelles, etc.).

CLE_UNIQUE Identifiant composite formé par la concaténation du matricule et du code organisme (`matricule||code_organisme`). Utilisé pour la jointure entre la Feuille 1 et la Feuille 2 des fichiers Excel.

CORRESPONDANCE_GRILLE_EMPLOI.xlsx
(Nouveau) Référentiel de correspondance entre libellé d'emploi et barème salarial, utilisé pour le fuzzy join dans l'étape 07.

F1 / F2 Les deux feuilles de chaque fichier Excel mensuel. F1 = données agents (identité, situation, montants). F2 = codes de rémunération élémentaires (composantes du salaire brut, identifiées par codes numériques).

Fuzzy join Jointure sur libellé normalisé (après suppression des accents, ponctuation, passage en majuscules). Utilisée pour rattacher les codes ANSTAT et les barèmes malgré les variations orthographiques.

Gold Couche de données finales prêtes à l'analyse : agrégats (masse salariale, effectifs) et panel individu×période enrichi.

GRADE Colonne ajoutée par l'étape 06b. Contient le grade fonctionnaire extrait ou imputé (ex: A3, B2, C1). Distinct de `grade` (libellé brut issu de F1).

GRADE_SOURCE
Traçabilité du grade : `OBSERVE` (extrait directement), `IMPUTE` (inféré statistiquement par cascade emploi), `NF_NON_IMPUTABLE` (non-fonctionnaire ou emploi sans historique).

Imputation en cascade
Méthode d'imputation du grade en trois niveaux de priorité : P1 = mode du grade pour cet emploi ce mois précis, P2 = mode pour cet emploi cette année, P3 = mode pour cet emploi sur toute la série.

GSBPM *Generic Statistical Business Process Model*. Cadre de référence international pour la production de statistiques officielles, développé par l'UNECE (version 5.2, 2024). Décompose toute chaîne de production en 8 phases : spécification, conception, construction, collecte, traitement, analyse, diffusion, évaluation. Adopté par l'INSEE, l'ONS UK, le Census Bureau et Eurostat. Voir le chapitre 8 pour la correspondance avec les scripts du pipeline.

Medallion Architecture
Architecture en couches Bronze / Silver / Gold inspirée du *Data Lakehouse*. Chaque couche améliore la qualité et la valeur des données. Complémentaire du GSBPM : la médaillon définit *comment stocker* les données, le GSBPM définit *quoi en faire*.

MinIO Serveur de stockage objet compatible S3, auto-hébergé. Accessible via le client boto3 de la même façon qu'AWS S3.

MMYYYY Format des noms de fichiers mensuels (ex: `012015` = janvier 2015). Utilisé comme identifiant de période dans le pipeline.

MODELE_ANSTAT
Feuille Excel au format hiérarchique présentant les statistiques de salaire brut (min/moyen/max) par secteur d'activité, dans la structure définie par l'ANSTAT pour la comparaison internationale.

Panel Base de données longitudinale où chaque ligne représente un agent à un mois donné (format *individu* × *période*).

Parquet Format de fichier colonne binaire, compressé (Snappy dans ce pipeline), optimisé pour l'analyse analytique. Lecture rapide par Polars.

Polars Bibliothèque Python de manipulation de DataFrames, alternative à pandas, avec API Lazy (scan/sink) permettant le streaming de grands volumes.

Silver Couche de données enrichies : types corrects (Float64 pour les montants), situations normalisées, codes ANSTAT rattachés, lignes "autre" supprimées.

Staging Couche d'atterrissage des fichiers sources. Zone de dépôt initiale et stockage des référentiels et rapports intermédiaires.

statut_fonctionnaire Colonne brute F1 contenant la description textuelle du grade (ex: "Catégorie A3 – Échelon 2"). La fonction `extraire_grade()` en extrait le grade via regex.

Winsorisation Écrêtage statistique des valeurs extrêmes. Dans ce pipeline, le REVENU_TOTAL est écrêté à P1/P99 dans chaque cellule `mois` × `grade` × `CITP` (avec repli sur `mois` × `grade` si effectif insuffisant). Le Gold n'est pas modifié : la winsorisation est appliquée en mémoire uniquement.